

만약에

내가 지금 신입이라면

2026년에 살아남는 신입은,
시키기 전에 스스로 돌아가는 사람이다.

만약에 내가 지금 신입이라면

“2026년에 살아남는 신입은, 시키기 전에 스스로 돌아가는 사람이다.”

들어가며

돈 이야기를 먼저 하려고 합니다. 현실적인 도움을 드려야 하는 자리에서, 예쁜 말은 의미가 없습니다. 우리는 결과를 본 뒤에 사람을 믿습니다. 쓸쓸하지만 그게 이 나라에서 신뢰가 만들어지는 순서죠. 그래서 가장 솔직한 자기소개를 하겠습니다.

저는 33살에 국비학원에서 코딩을 처음 배웠습니다. 순수 문과 비전공자였습니다. 멀쩡한 회사를 한 번도 다녀본 적 없는 사람이었습니다. 영문학과 철학을 전공했고, 카페와 공간 사업, 마케팅, 건강기능식품, 화장품, 코인 트레이딩까지 — 분야를 가리지 않고 살아왔습니다. 남들이 보면 방향입니다.

그런 사람이 1년 만에 판교 IT 대기업에 입사했습니다. 이력서 합격률을 1%에서 66%로 바꿨습니다. 연봉은 2천만원대에서 단번에 5천만원대가 됐습니다. 1년, 2년이 지나면서 지금은 7천만원을 훌쩍 넘겼습니다. 지금은 본업 연봉 외에 부수입이 그보다 많습니다. 웹 서비스 3개, 모바일 앱 2개를 운영하고 있고, 멘토링과 강의도 하고 있습니다.

연말정산 결과레포트

입력값

총급여(전직장포함)

7,423,170 원

월급날 검토값

총급여(전직장포함)

7,423,170 원

돈 이야기는 여기서 끝입니다.

제가 진짜 하고 싶은 이야기는 이겁니다. 저는 이제 마흔을 바라보는 나이가 됐습니다. 33살까지 멀쩡한 회사를 한 번도 안 다녀본, 남들 눈에는 한심해 보였을 사람이 지금은 개발이 너무 잘 맞고, 매일이 좋습니다. 특히 최근 AI 에이전트를 만나고 나서는 날개를 단 기분입니다. AI가 아니었으면 웹 서비스 3개, 앱 2개를 혼자 만들고 운영하는 건 불가능했습니다.

그런데 이 AI가 문제입니다. AI 양극화가 점점 극단으로 가고 있습니다. 쓰는 사람은 생산성이 폭발하고, 안 쓰는 사람은 그 격차가 벌어지는 줄도 모른 채 뒤처지고 있습니다. 신입들이 죽어나고 있습니다. 더 분노하고 안타까운 건, 그분들이 그걸 모른다는 것입니다.

대단한 이야기처럼 들릴 수도 있고, 운이 좋았다고 생각하실 수도 있습니다. 둘 다 맞고, 둘 다 틀립니다. 운이라고 부를 수 있는 건 딱 하나, 절박했다는 것입니다. 강아지 수술비가 필요했고, 카페 빛이 남아있었고, 서른이 넘은 비전공자가 갈 수 있는 곳이 많지 않았습니디. 선택지가 없었기 때문에 온 힘을 쏟을 수 있었습니다. 그게 운이라면 운입니다.

지금은 백엔드 개발자로 일하면서, 200명 이상의 1:1 이력서 멘토링을 해왔습니다. 비전공자, 국비 출신, 부트캠프 수료생, 이직 준비생. 수많은 분의 이력서를 보고, 고민을 듣고, 방향을 잡아드렸습니다.

오래 하다 보니 사람마다 결이 달랐습니다. 재료가 있는 분은 그걸 어떻게 포장하느냐의 문제였습니다. 자기 PR에 성공해서 빠르게 결과를 내는 분들이 있었습니다. 재료가 부족해도 열심히 해서, 긴 시간의 축에서 결국 이루는 분들도 있었습니다.

하지만 어떤 분들은 방향 자체를 못 잡았습니다. 성장하는 근력이 부족한 분들이었습니다. 뭘 해야 하는지를 몰라서가 아니라, 어디로 가야 하는지를 몰라서 멈춰 있었습니다.

그리고 정말 드물지만, 재료 자체가 없는 분들도 있었습니다. 2~3년 차인데 작은 SI, 솔루션 회사에서 완전한 레거시만 다뤘고, 그마저도 개발이라고 부르기 어려운 수준이었습니다. CS 기술자에 가까운 경력을 쌓은 분들. 그런 분들이 서비스 기업에 가고 싶다는 마음은 있는데, 방법을 몰라서 헤매고 있었습니다.

그래서 “그릿 모먼트”를 시작했습니다. 8주짜리 소수 정예 멘토링 프로그램입니다. 매주 오프라인으로 만나고, 주간 과제를 내고, 코드 리뷰와 모의 면접까지 합니다. 단순히 기술을 가르치는 게 아니라, 사고 전환부터 설계, 구현, 그리고 그 경험을 면접관에게 설득하는 서사까지 — 8주 동안 처음부터 끝까지 같이 만듭니다.

많은 인원을 받지 않습니다. 한 기수에 소수만 받습니다. 대신 그 안에서는 밀도 있게 갑니다. 그 결과, 멘티 중에는 실서비스 100명 이상의 사용자를 가진 프로젝트를 만든 분들이 나왔고, 실제 사업을 준비하는 분들도 생겼습니다. 서류 합격률이 70%를 넘기고, 일주일에 면접을 3~4개씩 보게 된 분들도 있었습니다.

1기, 2기, 3기, 4기. 기수를 거듭하면서 조금씩 알려졌습니다. 대기 예약이 꽉 찼습니다. 지금은 5기를 준비해야 하는데, 리소스가 없어서 못 하고 있는 상황입니다. 그런데 대기 문의는 계속 옵니다.

그 사이에도 신입들에게 시간은 없습니다. 대기하는 동안 시장은 바뀌고 있고, AI 격차는 벌어지고 있습니다. 직접 만나지 못하는 분들에게도 이 감각을 전달할 방법이 필요했습니다. 그릿 모먼트의 혜택을 받지 못하지만 너

무 간절한 분들, 그분들을 어떻게 도와줄 수 있을까. 고민하다가 하나의 질문이 떠올랐습니다. 그래서 이 책을 쓰기로 했습니다.

“만약에 내가 지금, 2026년에 다시 신입이라면 어떻게 할까?”

3년 전 제가 했던 방식 그대로는 안 됩니다. 시장이 바뀌었습니다. AI가 코드를 짜주는 시대에 “열심히 공부했습니다”만으로는 면접관의 눈에 띄지 않습니다. 하지만 동시에 변하지 않는 것도 있습니다. 절박함, 루틴, 실행력. 그건 어떤 시대에도 통합니다.

이 책은 그 두 가지를 모두 담으려고 합니다. 변하지 않는 근본과, 2026년이라는 시대가 요구하는 구체적인 전략. 둘 다 없으면 안 됩니다. 근본만 있으면 방향이 틀리고, 전략만 있으면 체력이 부족합니다.

그리고 이 책의 모든 이야기는 마지막에 한 문장으로 수렴합니다.

이 책은 선배가 후배에게 직접 말하는 톤으로 쓰였습니다.

격식체 대신 ~다 체(평서형)를 사용합니다.

Part 1. 판이 바뀌었다

시대마다 상위 1%를 가르는 키워드가 있다

개발자 채용 시장에는 시대마다 “이걸 아는 사람”과 “모르는 사람”을 나누는 키워드가 있다. 이걸 모르면 아무리 열심히 해도 시장이 원하는 방향과 어긋난다. 열심히 하는 건 전제 조건이지, 차별화가 아니다.

2022~2023년, 키워드는 객체지향, 테스트 코드, 아키텍처, DDD였다.

이 시기에 이력서에 “객체지향 설계 원칙을 적용해서 도메인을 분리했습니다”, “테스트 커버리지 80% 이상을 유지했습니다”라고 쓴 사람은 면접에 불렸다. 반대로 “Spring Boot로 CRUD 게시판 만들었습니다”만 쓴 사람은 서류에서 잘렸다. 같은 시간을 투자했는데 결과가 달랐다. 뭘 공부했느냐가 아니라, 시장이 원하는 걸 공부했느냐의 차이였다.

내가 이력서 합격률 1%에서 66%로 바꿀 수 있었던 이유도 이거다. 100번 넘게 떨어지면서 깨달았다. 내가 쓰고 싶은 걸 쓰는 게 아니라, 시장이 듣고 싶은 말을 써야 한다는 걸. 그 뒤 이력서를 완전히 뜯어고쳤다. “뭘 했다”에서 “왜 그렇게 결정했다”로 관점을 바꿨더니, 갑자기 서류가 통과하기 시작했다.

2024년, 키워드는 대용량 트래픽이었다.

“동시 접속 10만 명을 어떻게 처리할 것인가”, “메시지 큐를 왜 도입했는가”, “캐시 전략을 어떻게 설계했는가.” 이런 질문에 자기 경험으로 답할 수 있는 사람과 아닌 사람의 차이가 선명했다. 면접장에서 “카프카요? 네, 써봤습니다”와 “카프카를 도입한 이유는 이벤트 드리븐으로 서비스 간 결합도를 낮추기 위해서였고, 대안으로 SQS도 검토했지만 순서 보장이 필요해서 카프카를 선택했습니다”의 차이. 전자는 수백 명이 할 수 있는 말이고, 후자는 소수만 할 수 있는 말이었다.

2026년, 키워드는 AI다.

단순히 “ChatGPT 써봤습니다”가 아니다. AI를 도구로 써서 실제 문제를 풀어본 경험, AI와 협업해서 프로젝트를 만들어본 경험, AI가 생성한 코드를 읽고 판단할 수 있는 엔지니어링 역량. 이것이 2026년에 상위 1%를 가르는 기준이다.

문제는 대부분의 취준생이 이걸 모른다는 거다. 여전히 2022년 방식으로 공부한다. CS 공부하고, 코딩테스트 풀고, 이력서에 Spring Boot 프로젝트를 넣는다. 틀린 건 아니다. 하지만 그것만으로는 2026년에 눈에 띄지 않는다. 마치 2024년에 “저는 게시판 CRUD를 만들 수 있습니다”라고 말하는 것과 같다. 기본은 되지만, 그걸로 뽐히지는 않는다.

증거: 채용공고가 말하고 있다

말만 하면 안 믿을 테니, 증거를 보여주겠다. 2026년 2월 기준, 실제 채용공고에서 직접 발췌한 내용이다. 일부는 이미 마감된 공고도 있다. 그만큼 시장은 빠르게 움직이고 있다.

대기업 - 이미 AI를 기본값으로 본다

카카오페이 채용공고를 보자. 우대사항에 이렇게 적혀 있다.

“AI 도구(코드 리뷰, 로그 자동 분석, 테스트 자동 생성 등)를 활용해 개발 공수를 줄여본 경험이 있으면 좋아요.”

직무 소개

지원자격 (공통)

- Java/Kotlin 등 JVM 언어로 서버 로직을 개발하고, 프로덕션 장애를 직접 해결해본 경험이 필요해요.
- Spring Boot, Spring Cloud를 이용해 마이크로서비스 아키텍처를 설계하고, 장애 복구 시나리오를 구축해본 분을 환영해요.
- RDBMS 환경에서 쿼리 최적화나 인덱스 튜닝으로 성능을 개선해본 적이 있으면 좋아요.
- OOP, MSA 개념을 적극적으로 적용해 코드 품질과 유지보수성을 함께 높여본 역량이 있으면 좋아요.
- Docker, Kubernetes로 무중단 배포나 롤백을 진행하며 자원을 효율적으로 스케일링해본 일이 있으면 좋아요.
- AI 도구(코드 리뷰, 로그 자동 분석, 테스트 자동 생성 등)를 활용해 개발 공수를 줄여본 경험이 있으면 좋아요.

“있으면 좋아요”라고 했지만, 이건 사실상 “없으면 뒤흔친다”는 뜻이다. 대기업이 우대사항에 넣었다는 건, 조만간 자격요건으로 올라간다는 신호다.

네이버도 마찬가지다. N캘린더 팀 공고를 보자.

“Claude Code, Cursor 등 AI 도구를 적극 활용하여 개발 생산성을 향상시킨 경험을 보유하신 분”

Who We Are

- 캘린더, 알림, Keep, 마이 와 같은 네이버의 개인화된 도구 서비스를 운영 및 개발하는 조직입니다.
- 네이버의 에이전트 서비스 Agent N과 연계하여 네이버 회원을 위한 에이전트 시나리오를 제안하고 개발합니다.
- 안정적인 운영 및 신규 기능 개발을 위해 AI를 활용한 다양한 생산성 혁신을 시도하고 있습니다.
- 에이전트 시대에 중요성이 높아지고 있는 개인화 도구의 새로운 기술적·기능적 가능성을 MCP 인터페이스와 연계하여 탐색하고 실현할 수 있는 조직입니다.

Claude Code, Cursor. 구체적인 도구 이름이 채용공고에 등장하기 시작했다. “AI 관심 있는 분”이 아니라, “이 도구를 써서 성과를 낸 분”을 찾고 있다. 추상적인 관심이 아니라 구체적인 경험을 원한다는 뜻이다.

라인플러스는 한 단계 더 나간다.

“실제 서비스에 AI 기술을 적용한 경험이 있고 깊은 이해도를 가진 분” “Langchain, Langgraph 등 AI 프레임워크를 활용한 서비스 개발 경험”

우대사항

- 실제 서비스에 AI 기술을 적용한 경험이 있고 깊은 이해도를 가진 분
- LangChain, LangGraph 등 AI 프레임워크를 활용한 서비스 개발 경험
- Redis, MongoDB 등 NoSQL 데이터베이스를 활용한 대용량 서비스 운영 경험 있으신 분
- Kafka 및 Spring Cloud Data Flow를 이용한 실시간 데이터 스트리밍 및 처리 경험 있으신 분
- 글로벌 환경에서의 서비스 개발 및 운영 경험을 가지신 분
- DevOps 환경에 대한 이해와 Kubernetes, Docker 등 컨테이너 기술 활용 경험이 있으신 분
- 영어 또는 다국어 협업 환경에서의 업무 경험이 있으신 분

“써봤다” 수준이 아니다. 실제 서비스에 적용하고, AI 프레임워크를 다룰 줄 아는 사람을 원한다. 이건 이미 “AI를 쓸 줄 아느냐”가 아니라 “AI로 프로젝트를 만들 줄 아느냐”를 묻는 거다.

두나무는 AI를 쓰되 맹신하지 않는 태도를 본다. “AI 도구를 활용해서 생산성을 높이되, 결과를 완전하게 이해하고 스스로 검증하시는 분.” AI가 뱉어낸 코드를 그대로 커밋하는 사람이 아니라, 읽고 판단하고 고칠 수 있는 사람을 원한다는 뜻이다.

배달의민족(우아한형제들)도 명확하다. “AI를 활용하여 일하는 방식을 개선해 보신 분.” 단순히 AI를 써본 게 아니라, AI로 일하는 방식 자체를 바꿔본 경험을 묻고 있다.

대기업의 메시지는 일관된다. AI는 이미 선택이 아니다. 기본값이다.

유니콘 - AI Native를 선언하다

유니콘은 한 발 더 나간다. 무신사를 보자. 2026년 1월, 무신사는 아예 **AI Native Engineering Track**이라는 이름의 채용을 열었다.

무신사는

‘이미 완성된 개발자’보다

‘함께 성장할 수 있는 개발자’를 찾습니다.

AI로의 전환은 선택이 아닌 필수입니다.

기술은 빠르게 변하고, AI는 우리의 일하는 방식을 바꾸고 있습니다.

이 변화 속에서 우리가 찾는 사람은 다음과 같습니다.

- 새로운 기술에 호기심을 가지고 빠르게 학습하는 분
- AI 도구를 활용해 더 나은 코드를 작성하고 싶은 분
- 완벽한 실력보다 함께 성장하려는 태도를 가진 분
- 실제 서비스를 만들며 사용자에게 가치를 전달하고 싶은 분

“AI Agent를 활용한 Coding 경험과 Prompt Engineering 경험이 있고, 결과물의 한계와 적용지점을 명확하게 아는 분.” 코딩테스트도 두 번 본다. 기본 코딩 역량 한 번, AI Agent 활용 능력 한 번. 이건 단순히 AI를 우대하는 게 아니다. AI를 쓰는 능력 자체를 평가하겠다는 선언이다.

컬리는 더 직접적이다.

“생성형 AI를 활용하는 과제전형(사전과제 또는 코딩테스트)이 진행됩니다.”

주요 업무

- Payment 및 Billing 시스템 설계/개발/운영
- 정산 플랫폼 및 회계 연동 시스템 설계/개발/운영
- 캐시 및 적립금(포인트) 시스템 설계/개발/운영
- AML(Anti-Money Laundering, 자금세탁방지) 시스템 개발/운영 및 컴플라이언스 대응
- AI를 활용한 신기술 및 업무효율을 위한 연구/개발
- 신규 프로젝트 PM/PL (Project Manager/Project Leader) 역할 수행

자격 요건

- 백엔드 개발(Java/Kotlin, Spring Framework) 경력 5년 이상인 분
- AWS 클라우드 및 MSA 환경 아키텍처 설계/운영 경험을 보유한 분
- 결제, 정산, 회계, 선불 등 핀테크 시스템 개발/운영 실무 경험을 보유한 분
- SAP, ERP 등 재무/회계 시스템 및 외부 금융 솔루션 API 연동 경험을 보유한 분
- AI Agent, MCP (Multi-Party Computation) 서버 연동 및 활용 경험을 보유한 분

우대 사항

- AI 기술(LLM 등)에 대한 높은 관심 및 업무 활용 능력을 보유한 분
- 선불카드, 포인트 등 도메인 지식 및 관련 법규에 대한 이해가 있는 분

과제전형에서 AI를 쓰게 한다. AI 없이 코딩하는 시대는 끝났다고 공식적으로 선언한 거다. “LLM 등을 활용한 RAG 등 AI powered 검색/추천 서비스 개발 경험”도 요구한다. AI를 도구로 쓰는 것을 넘어, AI 기반 서비스를 만들 줄 아는 사람을 찾고 있다.

오늘의집(버킷플레이스)은 구체적인 도구 목록을 나열한다.

“GPT, Claude 등 LLM 기반 도구를 실제 업무에 적용해보고, 그 과정에서 얻은 인사이트나 한계를 설명할 수 있는 분” “n8n, Langchain, LlamaIndex 등 AI 연동 및 자동화 도구 사용 경험이 있는 분”

‘당연한 불편’을 없애고 새로운 기준을 세우는 일은 쉽지 않지만, 그래서 더 보람 있는 일입니다. Interior & Life 서비스팀은 실패를 두려워하지 않고, 기존의 당연함에 질문을 던지며, 업 자체를 바꾸는 경험을 함께할 도전적인 동료를 기다립니다. 또한, 오늘의집은 AI Native 조직으로 변화 중입니다. 우리는 AI를 도구처럼 쓰는 것을 넘어, 일의 방식 자체를 함께 바꿔나갈 동료를 기다리고 있습니다.

 **AI와 동료처럼 함께 일하고 싶은 개발자라면, 바로 오늘의집이 당신의 Playground 입니다.**

인사이트와 한계를 설명할 수 있는 분. 핵심은 “설명할 수 있는”이다. 써봤다는 건 누구나 할 수 있다. 왜 쓰고, 어디서 한계를 느끼고, 어떻게 보완했는지를 자기 말로 설명할 수 있느냐. 그게 차이다.

당근은 아예 업무 설명에 “AI를 활용해 생산성을 극대화해요. Claude Code, Cursor 등 AI 도구로 반복 작업을 자동화하고 개발 속도를 높여요”라고 적어놨다. 우대사항이 아니라 일하는 방식 자체가 AI 기반이라는 뜻이다. 토스페이먼츠는 “기술 부문의 제약 없이 특정 문제를 끝까지 파고들어 문제를 해결해본 경험”을 원한다. 기술이 아니라 문제 해결의 끈기를 본다. 채널톡은 “문제 해결을 위해 Trade-off를 명확히 이해하고, 당면한 과제에 최적의 도구를 선택할 수 있는 분”을 찾는다. AI가 만능이 아니라는 걸 알고, 언제 쓰고 언제 안 쓸지 판단할 줄 아는 사람을 원하는 거다.

유니콘의 메시지는 더 강렬하다. AI를 쓸 줄 아느냐가 아니라, AI 시대에 일할 줄 아느냐를 묻고 있다.

스타트업 - 더 노골적이다

스타트업은 돌려 말하지 않는다. 나만의닥터(메라키플레이스) 공고를 보자.

“AI 레버리지 성장: 최근 6개월~1년간 AI 도구를 활용해 폭발적 생산성/역량 성장을 만들어 오고 계신 분.”

- Merakiplace의 AI Native Back-end Engineer는 고객이 가장 쉽고 편리하게 건강을 케어할 수 있는 Healthcare Super App을 만들기 위해 <나만의닥터> 서비스의 Backend를 이루는 다양한 서버들과 인프라를 개발/운영해 나가고 있습니다.
- AI Native Engineering 시대에도 가치를 극대화하고, 더 큰 임팩트를 만들어 내기 위해 AI를 적극 활용하여 최근 6개월~1년간 폭발적인 생산성향상과 역량 성장을 만들어 오고 계신 분을 찾고 있습니다.
- 모든 구성원들이 스스로 문제를 정의하고, 이를 Product로 해결해가면서 팀, 제품, 그리고 구성원이 함께 성장해 나가는 것을 추구합니다.
- AI를 레버리지하여, Legacy 시스템의 문제를 공격적으로 해결해 나가며, 과거보다 높은 속도와 품질을 유지해 나갈 수 있는 시스템을 만들어 나가는 것에 큰 흥미를 가지고 계신 분과 함께 하고 싶습니다

주요업무

- AI Native Engineering을 통한 Healthcare 혁신
- Healthcare Super App 성장 과정의 핵심 Backend 문제들을 AI를 적극 활용하여 해결
- AI 도구들(Cursor/Claude/Gemini/MCP)을 활용한 개발 속도와 품질의 향상
- Engineering 을 통한 고객 / 제품 / 팀 / 동료 의 문제를 탁월하게 해결
- Python/Django Legacy 시스템 기반에서 Go/Kotlin/Java/Typescript 등을 전략적으로 선택하여 최적 솔루션 구현
- Sprint 기반 제품 개발
- Chapter/Squad 소속으로 문제 정의부터 배포까지 전 과정 참여
- Why/What(문제 정의) When/How(계획 및 시스템 디자인) 실행 및 운영의 완전한 오픈십
- 모니터링과 분석을 자동화를 통한 안정적인 서비스 운영
- 0to1 단계의 팀/제품/기능: 빠른 검증과 민첩한 실행
- AI/Engineering 를 통한 고객 문제 발견 및 검증 프로세스 구축
- 실험 친화적 아키텍처 설계 (만들기 쉽고, 없애기도 쉬운 모듈화)
- 보안과 속도를 모두 확보하는 엔지니어링 기반 구축
- 1to10 단계의 팀/제품/기능: 확장성과 안정성 확보
- 신뢰성, 확장성, 성능 최적화를 통한 시스템 진화
- 조직 성장에 따른 개발 생산성 병목 해결 (빌드, 배포, 협업 최적화)
- AI 기반 자동화를 통한 팀 생산성 극대화
- 10to100 단계의 팀/제품/기능: 대규모 서비스 운영
- 대용량 트래픽 및 데이터 처리 아키텍처 설계
- 높은 신뢰성 확보와 동시에 지속적인 실험 지원
- 전사적 엔지니어링 표준 및 플랫폼 구축

“폭발적 생산성 성장.” 이보다 직설적일 수 없다. AI를 써서 실제로 성장한 궤적을 보여달라는 거다. 6개월~1년이라는 기간까지 명시했다. 면접에서 “최근 6개월간 AI로 뭘 했냐”고 물었을 때 할 말이 없으면 끝이다.

고위도도 마찬가지다.

“AI를 이용하여 본인 또는 팀의 업무 효율성을 증대시킨 경험이 있으신 분”

우대사항

- 컴퓨터 공학 또는 관련 분야의 학사 학위 이상인 분
- 비즈니스, 프로덕트, 개발자 등 다양한 이해관계자들과 합리적이고 유연한 커뮤니케이션이 가능한 분
- 레거시 프로젝트를 신규 프로젝트로 리팩토링 한 경험이 있으신 분
- AI 를 이용하여 본인 또는 팀의 업무 효율성을 증대 시킨 경험이 있으신 분

개인이든 팀이든, AI로 실제 효율을 올려본 경험을 원한다. “써봤다”가 아니라 “효율을 올렸다”는 결과를 원하는 거다.

버즈빌은 아예 지원 도구를 나열한다. “AI 톨 적극 지원(Claude Code Max \$200 플랜, ChatGPT, Cursor, GitHub Copilot, Coderabbit).” 회사가 월 20만 원이 넘는 AI 도구 비용을 개발자 한 명당 지불한다는 건, AI가 그만큼의 생산성을 만들어낸다고 확신하기 때문이다.

포스타입은 “반복 업무의 자동화 및 AI 도구 활용을 통한 효율화”를 강조한다. 더빌딩은 한 발 더 나가서 아예 **바 이브코더**를 모집한다. “AI 생성 기술은 이제 ‘보조 도구’가 아니라 개발 방식 자체를 바꾸는 핵심 요소라고 생각합니다.” 스타트업일수록 AI에 대한 태도가 더 적극적이다. 여유가 없기 때문이다. 적은 인원으로 최대 성과를 내야 하니, AI로 레버리지를 만들 줄 아는 사람이 절실하다.

대기업, 유니콘, 스타트업. 규모는 다르지만 메시지는 같다. **AI는 이미 채용의 기본 조건이 되었다.** 아직도 “AI는 나중에 배워도 되겠지”라고 생각하는 사람이 있다면, 지금 이 공고들을 직접 찾아가서 읽어봐라. 시장은 이미 움직였다.

시대의 키워드를 읽는 사람이 판을 가져간다. 읽지 못하는 사람은 열심히 해도 방향이 틀린다. 그리고 방향이 틀린 노력은 노력한 사람을 지치게 만든다.

채용공고에는 행간이 있다

채용공고를 그냥 읽지 마라. 그건 번역이 필요한 문서다.

“주도적으로 문제를 해결하는 분”이라고 쓰여 있으면, 진짜 의미는 이거다: **“시키지 않아도 알아서 하는 사람을 원한다. 물어보기 전에 찾아보는 사람을 원한다.”** 왜 그런 문구가 들어갔는지 생각해봐라. 그 팀에 시키는 것만 하는 사람이 있어서 그렇다. 팀장이 매번 하나하나 알려줘야 하는 상황에 지쳐서 그 문구를 넣은 거다.

“AI/ML 경험 우대”라고 쓰여 있으면, 진짜 의미는 이거다: **“우리 팀에 AI를 실무에 적용할 줄 아는 사람이 없다. 누군가 와서 해줬으면 좋겠다.”** 이건 기회다. 현업에서 AI를 써본 사람이 아직 많지 않기 때문에, 주니어라도 AI를 실제로 활용해본 경험이 있으면 그 자체로 가치가 있다.

“빠르게 성장하는 환경”이라고 쓰여 있으면: **“일이 많고 체계가 없다. 알아서 헤쳐나갈 수 있는 사람만 살아남는다.”** 이걸 보고 겁먹을 필요는 없다. 오히려 주니어에게는 기회일 수 있다. 체계가 없다는 건 내가 체계를 만들 수 있다는 뜻이고, 그게 곧 성과가 된다.

“커뮤니케이션 능력이 뛰어난 분”이라고 쓰여 있으면: **“이전에 커뮤니케이션이 안 되는 사람 때문에 고생한 적이 있다.”** 기술만 잘하면 된다고 생각하는 사람이 놓치는 부분이다.

하나 더 보여주겠다. 행간 읽기를 실제로 연습해보자.

토스뱅크 공고에 이런 문장이 있다.

“주어진 요구를 그대로 구현하기보다, 스스로 기준을 세우고 마주한 문제를 구조적으로 개선해 본 경험이 필요해요.”

이런 분과 함께하고 싶어요

- Kotlin/Java로 Spring Framework 기반의 서비스를 개발한 경험이 있는 분이 필요해요.
- 비즈니스 요건을 빠르게 이해하고 능숙하게 데이터 모델과 API를 디자인할 수 있는 분이 필요해요.
- 변화를 두려워하지 않고 지속적인 성장을 원하시는 분이면 잘 맞으실 거예요.
- 동료에게 필요한 지식을 전파하고 협업을 즐기시는 분이면 좋아요.
- 대용량 트래픽을 안정적으로 운영해본 경험이 있는 분이면 좋아요.
- Redis와 Kafka를 다뤄본 경험이 있는 분이면 좋아요.

이력서에는 아래 내용을 꼭 포함해 주세요.

- 실제 본인이 참여한 프로젝트와 기여 정도, 어려운 과제를 극복한 과정이 드러날 수 있는 구체적인 사례가 최소 1가지 이상 필요해요.
- 서비스 개발을 담당하는 만큼, 문제 해결 역량을 중요하게 생각하고 있어요. 마주한 문제를 정의하고 이를 해결하기 위해 어떤 시도를 했는지, 나아가 그 결과는 어땠는지 등이 확인될 수 있도록 문제 해결의 과정을 상세하게 적어주세요.
- 대규모 트래픽을 다뤄보신 경험 또는 기존 대비 더 나은 결과를 위해 개선을 시도해 보신 경험이 있다면 꼭 공유해 주세요.
- 개발자로서 어떤 성장을 생각하고 계시고, 성장을 위해 어떤 노력을 하고 계신지 적어주세요.

표면만 읽으면 “아, 문제 해결 능력을 원하는구나”로 끝난다. 하지만 행간을 읽어봐라. “주어진 요구를 그대로 구현하기보다”라는 전제가 붙어 있다. 이건 뒤집어 말하면, 시키는 대로만 하는 사람은 필요 없다는 뜻이다. 기획서를 받아서 코드로 옮기는 건 시도 한다. 토스뱅크가 원하는 건 **기능을 구현하는 사람이 아니라, 문제를 정의하는 사람**이다. 이게 바로 “기능 구현자”에서 “문제 해결자”로의 패러다임 전환이다.

강남언니 공고에는 더 흥미로운 문장이 있다.

“문제 해결에 필요하다면 자신의 지식과 경험을 내려놓을 수 있는 분”

자격요건

- 웹 서버 엔지니어링 경험과 역량을 갖춘 분
- 하나 이상의 OO(Object-Oriented) 프로그래밍 언어를 능숙하게 다루는 분
- 문제의 본질을 이해하고 답을 찾기 위해 동료의 의견에 집중하는 분
- 자신의 생각을 말, 글, 그림을 통해 합리적으로 표현할 수 있는 분
- 문제 해결에 필요하다면 자신의 지식과 경험을 내려놓을 수 있는 분
- 지금보다 더 나은 엔지니어로 성장하고 싶은 분

“자신의 지식과 경험을 내려놓을 수 있는 분.” 이 문장을 곱씹어봐라. 왜 이런 문구가 채용공고에 들어갔을까? 기존 방식을 고집하는 사람 때문이다. “나는 이렇게 해왔으니까 이게 맞아”라고 버티는 사람. AI 시대에 이 태도는 치명적이다. 6개월 전의 베스트 프랙티스가 오늘은 아닐 수 있다. 새로운 도구, 새로운 방법론, 새로운 패러다임이 쏟아지는 시대에 과거의 성공 경험에 집착하는 건 오히려 발목을 잡는다. 강남언니가 원하는 건 **배운 것을 버릴 줄 아는 유연함**이다.

이처럼 채용공고는 단어 하나하나에 그 회사의 사정이 담겨 있다. 눈에 보이는 문장 뒤에 숨어 있는 의도를 읽을 줄 알아야 한다.

채용공고를 100개 읽어봐라. 진짜로 100개. 한 분야만이 아니라 다양한 회사, 다양한 규모, 다양한 도메인의 공고를 읽어라. 그러면 패턴이 보인다. 어떤 키워드가 반복되는지, 어떤 기술 스택이 떠오르는지, 어떤 역량을 강조하는지. 그게 시장이 지금 말하고 있는 것이다. 시장은 늘 말하고 있다. 듣는 사람이 없을 뿐이다.

나는 멘토링을 하면서 멘티들에게 항상 이걸 먼저 시킨다. 채용공고 분석. 대부분이 귀찮아한다. “그냥 뭘 공부하면 되는지 알려주세요.” 이게 이미 문제다. 누군가한테 답을 받아서 공부하는 사람은, 면접에서도 답을 받아서 말하는 티가 난다. 면접관은 그걸 1분 만에 알아챈다. “이 사람은 자기 머리로 생각한 게 아니라 어디서 들은 걸 말하고 있구나.”

스스로 시장을 읽은 사람은 다르다. “왜 이걸 공부했나”고 물으면 자기 말로 답한다. “채용공고를 분석해보니 이런 패턴이 보였고, 그래서 이 기술을 선택했습니다.” 이 한 마디가 면접관에게 주는 신호는 강력하다. “이 사람은 시장을 읽을 줄 안다. 시키지 않아도 방향을 잡을 줄 안다.” 그 차이가 합격과 불합격을 가른다.

백캐스팅 — 미래에서 역추적하는 사고법

대부분의 사람은 현재에서 출발한다. “지금 뭘 할 수 있지?” 여기서 시작하면 시야가 좁아진다. 할 수 있는 것만 하게 된다. 오늘 할 수 있는 건 어제와 비슷하고, 내일도 비슷할 거다. 이러면 6개월이 지나도 크게 달라지지 않는다.

백캐스팅은 반대다. **도착점을 먼저 정하고, 거기서 역추적한다.**

“6개월 뒤 나는 이 회사의 백엔드 개발자로 일하고 있다.”

이걸 먼저 박아놓고, 역으로 질문한다.

- 그 회사에 붙으려면 이력서에 뭐가 있어야 하는가?
- 그 이력서를 만들려면 어떤 프로젝트가 필요한가?
- 그 프로젝트를 하려면 어떤 기술이 필요한가?
- 그 기술을 배우려면 이번 달에 뭘 해야 하는가?
- 이번 주에 뭘 해야 하는가?
- 오늘 뭘 해야 하는가?

이렇게 내려오면 오늘 할 일이 명확해진다. “뭘 해야 할지 모르겠다”는 말이 사라진다. “이것도 해야 하고 저것도 해야 하는데 시간이 없다”는 말도 사라진다. 도착점이 명확하면 우선순위가 자동으로 정해지니까.

나도 이 방식을 썼다. 국비학원을 다니면서 “6개월 뒤 판교 IT 대기업”을 도착점으로 잡았다. 주변에서는 “국비 출신이 대기업을? 현실적으로 생각해”라고 했다. 그런데 도착점을 낮추면 오늘의 노력도 낮아진다. “중소기업이면 되지”라고 생각하면 하루 8시간으로 충분하다. “대기업 간다”고 생각하면 하루 16시간을 투자할 수 있다. 높게 잡았기 때문에 학원 커리큘럼에 안주하지 않고 방통대를 병행할 수 있었다.

멘토링에서 자주 보는 패턴이 있다. 도착점을 안 정하는 사람들이다. “일단 공부하고 있어요.” 뭘 위해서? “좋은 회사에 가려고요.” 어떤 회사? “아직 잘 모르겠어요.” 이러면 공부의 방향이 매번 바뀐다. 이번 주는 Spring, 다음 주는 Node.js, 그다음 주는 “요즘 Go가 뜬다던데.” 6개월이 지나면 이것저것 조금씩 건드려봤지만 깊이 있는 게 하나도 없는 상태가 된다.

백캐스팅의 핵심은 “될까 안 될까”를 고민하는 시간을 없애는 것이다. 일단 도착점을 정해버리면, 남은 건 “어떻게”뿐이다. “어떻게”는 행동으로 풀 수 있는 문제다. “될까 안 될까”는 아무리 고민해도 답이 안 나오는 문제다. 답이 안 나오는 문제에 시간을 쓰지 마라.

Part 2. 문제는 시스템이다

절박함은 감정이 아니라 시간표다

“절박합니다.” 멘토링에서 가장 많이 듣는 말이다. 취업 준비 중인 사람, 이직을 고민하는 사람, 부트캠프를 막 수료한 사람. 다들 절박하다고 한다.

그런데 시간표를 보면 절박하지 않다.

멘토링 중에 만난 한 멘티가 있었다. 비전공자, 부트캠프 수료, 3개월째 취업 준비 중. 첫 미팅에서 말했다. “정말 절박합니다. 뭐든 하겠습니다.” 그래서 물었다. 하루 루틴이 어떻게 되냐고.

12시에 일어난다. 밥 먹고 1시쯤 공부를 시작한다. CS 공부를 좀 하다가, 코딩테스트를 풀다가, 이력서를 좀 고치다가, 5시쯤 끝낸다. “오늘은 좀 집중이 안 돼서.” 주 25시간. 나머지 시간은 유튜브를 보거나, 취업 카페에서 다른 사람들의 불안한 글을 읽거나, “오늘은 좀 쉬자”를 한다.

절박한 사람의 시간표가 이런가?

내가 국비학원을 다닐 때, 하루가 이랬다.

아침에 일어난다. 학원에 간다. 학원이 5분 거리니까 늦을 일이 없다. 수업이 시작되기 전에 어제 복습한 걸 한번 더 본다. 수업을 듣는다. 쉬는 시간에 알고리즘을 푼다. 다른 사람들이 커피 마시러 나갈 때, 나는 자리에 앉아서 문제 하나를 더 푼다. 수업이 끝나면 바로 복습에 들어간다. 복습이 끝나면 패스트캠퍼스 강의를 본다. 학원에서 안 가르쳐주는 것들, 시장에서 원하는 것들을 여기서 채운다. 그게 끝나면 알고리즘을 더 푼다. 자기 전에 방통대 과제를 한다.

하루 16시간 이상. 주 6일. 6개월.

내 원칙은 딱 세 개였다. 안 늦기. 안 빠지기. 남들보다 1시간 더 하기.

복잡한 거 아무것도 없다. 이 세 개를 6개월 동안 지켰을 뿐이다. 특별한 재능이 필요하지 않다. 남들이 쉬는 때 1시간 더 앉아있는 거다. 하루에 1시간은 별것 아닌 것 같지만, 6개월이면 180시간이다. 180시간은 일반적인 사람이 한 달 풀타임으로 공부하는 양이다. 그 차이가 서류 합격과 불합격을 가른다.

저 멘티에게 말했다. “지금 시간표로는 안 됩니다. 절박하다면서 시간표가 절박하지 않아요.” 기분 나쁠 수 있는 말이다. 하지만 3개월 뒤에도 같은 자리에 있는 것보다는 낫다.

절박함을 입으로 말하는 사람과, 시간표로 증명하는 사람. 수많은 상담을 하면서 이 둘의 결과는 확연히 달랐다. 입으로 말하는 사람은 6개월이 지나도 여전히 “절박합니다”를 반복하고 있었다. 시간표로 증명한 사람은 이미 회사에 다니고 있었다.

결과를 만드는 건 감정이 아니라 시간이다. 시간을 어디에 얼마나 쓰고 있는지. 그게 진짜 절박함의 척도다.

환경이 의지를 이긴다

의지력은 소모품이다. 매일 아침 “오늘도 열심히 해야지”라고 다짐하는 순간, 이미 에너지를 쓰고 있다. 그 에너지는 한정되어 있다. 오전에 다짐하느라 쓰고, 오후에 유혹을 참느라 쓰고, 저녁에는 바닥이 나서 “내일부터 다시 하자”가 된다. 이것 매일 반복하면 3일 만에 무너진다.

나는 의지 대신 환경을 바꿨다.

국비학원에 다닐 때, 집에서 학원까지 왕복 4시간이었다. 화성에서 가산디지털단지까지. 아침에 일어나서 버스를 타고, 지하철을 갈아타고, 또 버스를 타야 했다. 매일 4시간을 길에서 버렸다. 버스에서 강의를 들으려고 했지만, 솔직히 멀미 나고 졸리고 집중이 안 됐다. 한 달을 그렇게 버텼다.

그러다 계산을 해봤다. 왕복 4시간 × 주 5일 × 4주 = 월 80시간. 80시간이면 패스트캠퍼스 강의 하나를 완강할 수 있는 시간이다. 이것 길에서 버리고 있다.

결심했다. **학원 옆으로 이사했다.** 화성에 있는 원룸 계약을 깨고, 가산동 근처에 새 방을 잡았다. 위약금이 나갔다. 이사 비용이 들었다. 근데 그건 투자였다.

그 순간 하루에 4시간이 생겼다. 의지로 “일찍 일어나야지” 하는 게 아니라, 물리적으로 학원이 5분 거리에 있으니 늦을 수가 없었다. 체력 소모가 줄었다. 이동하느라 쓰던 에너지를 공부에 쓸 수 있었다. 밤늦게까지 학원 근처 카페에서 공부하다가 걸어서 집에 갈 수 있었다.

환경 설계는 이런 거다.

“운동해야지”가 아니라 헬스장 옆으로 이사하는 것. “공부해야지”가 아니라 카페에 노트북을 놓고 퇴근 후 바로 가는 루트를 만드는 것. “유튜브 안 봐야지”가 아니라 유튜브 앱을 삭제하는 것. “집에서 집중이 안 돼”라면 집에서 공부하지 마라. 카페로 가라. 도서관으로 가라. 스터디 공간을 빌려라. 환경을 바꿔라.

멘토링에서 만난 사람 중에 “의지가 약해서 공부를 못하겠다”는 사람이 많았다. 의지가 약한 게 아니다. 환경이 나쁜 거다. 스마트폰이 옆에 있고, 침대가 보이고, TV가 켜져 있는 상태에서 공부에 집중하라는 건, 패스트푸드점에서 다이어트하라는 것과 같다. 의지의 문제가 아니라 구조의 문제다.

의지에 기대면 3일을 버틴다. 환경을 바꾸면 6개월을 간다. 그리고 6개월이면 인생이 바뀐다. 내가 증거다.

2026년에는 환경 설계에 한 가지가 추가됐다. 개발 환경 자체를 AI 중심으로 세팅하는 것이다. 에디터를 열면 AI가 바로 붙어 있는 상태. Claude Code가 터미널에 상주하는 상태. 막히면 3초 안에 물어볼 수 있는 상태. 이게 환경이다. 북마크만 해둔 사람과, 이미 세팅하고 매일 쓰는 사람은 6개월이면 확연히 갈린다. 의지로 “AI를 공부 해야지” 하는 게 아니라, AI가 이미 내 작업 흐름 안에 있게 만드는 것. 이것도 환경 설계다.

작고 확실한 원칙이 루틴을 만든다

거창한 계획은 3일 만에 무너진다.

“매일 알고리즘 5문제, 강의 3시간, 프로젝트 2시간, 블로그 1편, CS 정리 1챕터.” 이런 계획을 세워본 적 있을 거다. 첫날은 의욕이 넘쳐서 다 한다. 둘째 날은 어제 피곤해서 알고리즘을 3문제밖에 못 풀고, 블로그는 내일로 미룬다. 셋째 날은 “어제도 다 못 했는데”라는 자괴감에 전부 포기한다. 넷째 날은 계획을 새로 세운다. 이 사이클을 매주 반복한다.

루틴은 단순해야 산다.

거창한 계획이 무너지는 이유는, 지킬 게 너무 많기 때문이다. 내가 세 개의 원칙만 세웠던 것도 같은 이유다. 뭘 공부할지, 얼마나 할지는 그날그날 달라도 된다. 컨디션이 좋은 날은 알고리즘을 10문제 풀어도 되고, 안 좋은 날은 강의만 들어도 된다. 하지만 원칙은 안 깨뜨린다. 단순하면 지키기 쉽다. 지키기 쉬우면 오래 간다. 오래 가면 결과가 나온다.

국비학원은 솔직히 최악이었다. 커리큘럼은 엉망이고, 강사의 수준도 들쭉날쭉했다. 시장에서 원하는 것과 가르치는 것의 괴리가 컸다. 대부분의 사람은 불평했다. “학원이 별로야”, “강사가 안 가르쳐줘”, “커리큘럼이 시장이랑 안 맞아.”

맞는 말이다. 근데 그래서? 불평한다고 학원이 좋아지나? 강사가 바뀌나? 안 바뀐다. 불평에 쓰는 에너지를 차라리 공부에 쓰는 게 낫다.

나는 불평 대신 시스템을 만들었다. 학원 커리큘럼이 부족하면 직접 채운다. 팀원들과 모여서 자체 커리큘럼을 짰다. “학원에서 안 가르쳐주는 것 리스트”를 만들고, 각자 하나씩 맡아서 공부하고, 서로 가르쳐줬다. 쉬는 시간에 알고리즘을 풀었다. 수업이 끝나면 패스트캠퍼스를 들었다. 주말에는 방통대 과제를 했다. 학원은 뼈대일 뿐이고, 살은 내가 붙였다.

이게 루틴의 힘이다. 루틴은 “매일 뭘 할지”를 정하는 게 아니다. **“어떤 상황에서도 이것만은 한다”를 정하는 거다.** 학원이 좋은 나쁜든, 컨디션이 좋은 나쁜든, 안 늦고, 안 빠지고, 1시간 더 한다. 이 단순한 원칙이 6개월을 버텼다.

환경을 탓하는 사람은 환경이 좋아져도 탓할 거리를 찾는다. 시스템을 만드는 사람은 환경이 나빠도 돌아간다. 그리고 회사는 후자를 뽑는다. 왜냐하면 회사도 완벽한 환경이 아니니까. 레거시 코드가 있고, 문서가 없고, 일정은 빡빡하고, 요구사항은 수시로 바뀐다. 그 안에서 돌아가는 사람. 그게 회사가 원하는 사람이다.

Part 3. 무기는 세 개다

2026년에 취업하려면 무기가 하나로는 부족하다. 세 개의 축을 동시에 돌려야 한다.

엔지니어링 기초는 체력이다

CS, 객체지향, 테스트 코드, 아키텍처. 이런 것들이다.

“AI 시대에 이런 거 아직도 중요해?” 이 질문을 멘토링에서 자주 받는다. 중요하다. 오히려 더 중요해졌다.

이유를 말하겠다. AI가 코드를 생성하는 시대일수록, **그 코드가 맞는지 틀린지 판단하는 능력**이 필요하다. AI한테 “사용자 인증 시스템 만들어줘”라고 하면 똑딱 만들어준다. 근데 그 코드에 보안 취약점이 있는지, 확장성 문제가 있는지, 우리 아키텍처에 맞는지를 누가 판단하나? AI가 제안하는 아키텍처가 우리 상황에 적합한지 평가할 수 있어야 한다. 그 판단력은 기초에서 나온다.

AI가 “Redis를 쓰세요”라고 제안했을 때가 대표적이다. 왜 Redis인지, Memcached와 뭐가 다른지, 우리 데이터 구조에서 Redis의 자료형이 적합한지, 영속성이 필요한 상황인지, 클러스터링은 어떻게 할 건지. 이 질문들에 답할 수 없으면 AI의 제안을 그대로 따르는 것밖에 못한다. 그건 “AI를 쓰는 것”이 아니라 “AI에 쓰이는 것”이다.

멘토링에서 봤던 패턴이 있다. 이력서에 “Redis 썼습니다”라고 쓰는 사람은 많다. 면접에서 “왜 Redis를 선택했냐”고 물으면 “인강에서 Redis를 쓰길래 따라했습니다.” 이런 답이 나온다. 반면에 “Redis, Memcached, 로컬 캐시를 비교했고, 우리 케이스에서는 데이터 구조의 다양성이 필요해서 Redis를 선택했습니다. 다만 단일 장애점 문제를 고려해서 Redis Sentinel을 구성했습니다.” 이렇게 답하는 사람은 거의 없다. 전자는 스펙이고, 후자가 역량이다. 그 차이는 기초에서 나온다.

다만, 이건 단기간에 되는 게 아니다. 벼락치기로 한 달 동안 CS 몰아서 외우는 건 면접에서 바로 들통난다. “힙과 스택의 차이가 뭔가요?”에 외운 답을 말할 수는 있지만, “그러면 이런 상황에서 메모리 이슈가 생기면 어떻게 접근하시겠어요?” 같은 후속 질문에서 무너진다.

하루에 1시간, 매일 조금씩 쌓는 거다. 코딩테스트 한 문제. CS 한 주제 깊이 파기. 테스트 코드 한 케이스 작성해 보기. 설계 패턴 하나 적용해보기. 이것 루틴으로 돌린다. 한 달 뒤에는 티가 안 나지만, 3개월이면 차이가 보이고, 6개월이면 확연히 다르다.

체력이 없으면 기술을 써도 제대로 못 쓴다. 마라톤을 뛰려면 기초 체력이 있어야 하듯, 좋은 소프트웨어를 만들려면 엔지니어링 기초가 있어야 한다. 이건 유행이 아니라 토대다. AI 시대가 와도, 그다음 시대가 와도, 이 토대는 변하지 않는다.

AI 엿지는 스스로 찾는 것이다

2026년에 “AI 잘 쓰는 사람”의 정의가 바뀌었다. 바이브코딩으로 앱 하나 똑딱 만드는 건 더 이상 특별하지 않다. 누구나 한다. ChatGPT한테 “쇼핑몰 만들어줘” 하면 만들어준다. 그런 포트폴리오는 면접관 입장에서 의미가 없다. 그건 AI를 쓴 거지, AI를 활용한 게 아니다.

진짜 엿지는 최전선에서 스스로 찾는 것이다.

트위터를 보라. 링크드인을 보라. 유튜브를 보라. 지금 이 순간에도 AI 도구가 쏟아지고 있다. 새로운 모델이 나오고, 새로운 프레임워크가 등장하고, 새로운 적용 사례가 공유되고 있다. 해외 개발자들이 “이렇게 써봤더니 이런 결과가 나왔다”를 실시간으로 공유하고 있다. 이걸 미친 듯이 흡수하는 게 첫 번째다.

매일 30분은 AI 관련 콘텐츠를 소비해라. 트위터에서 AI 관련 계정 20개를 팔로우해라. 링크드인에서 AI 엔지니어들의 글을 읽어라. 유튜브에서 새로운 도구 리뷰를 보라. 이건 공부가 아니라 정보 수집이다. 인풋 없이 아웃풋은 없다.

두 번째는 직접 시도하는 거다. 읽기만 하면 아무 일도 안 일어난다.

Claude Code를 써보라. 프로젝트 하나를 처음부터 끝까지 Claude Code와 함께 만들어보라. Cursor를 써보라. Copilot을 써보라. 하나를 쓰다가 막히면 다른 걸 시도해보라. 이 과정에서 시행착오가 생긴다. “아, 이 도구는 이런 건 잘하는데 저런 건 못하는구나.” “이런 프롬프트를 쓰면 결과가 더 좋구나.” “AI가 생성한 코드에서 이런 패턴의 버그가 자주 나오는구나.” 이 시행착오가 자산이다. 왜냐하면 이건 직접 해봐야만 아는 것이니까.

“AI가 코드를 다 짜주는데 개발자가 왜 필요해?” 이런 질문은 AI를 제대로 써보지 않은 사람이 한다. 실제로 프로젝트에 써보면 안다. AI는 시키면 한다. 하지만 뭘 시킬지는 사람이 정해야 한다. 어떤 문제를 풀어야 하는지, 어떤 방향으로 가야 하는지, AI가 만든 결과물이 맞는지 틀린지, 이건 여전히 사람의 영역이다. 더 정확히 말하면, AI를 써봤기 때문에 아는 사람의 영역이다.

내가 지금 신입이라면 이렇게 할 거다.

매일 30분은 AI 관련 콘텐츠를 소비한다. 그리고 주 1회는 반드시 뭔가를 직접 만들어본다. 작은 거라도 상관없다. AI로 API 하나 만들어보기, AI로 테스트 코드 생성해보기, AI로 코드 리팩토링해보기. 이걸 반복하면 “AI를 쓸 줄 아는 사람”과 “AI로 문제를 풀어본 사람”의 차이가 생긴다.

그리고 그 과정에서 발견한 것을 기록한다. “Claude Code로 이벤트 드리븐 아키텍처를 구현해봤더니 이런 한계가 있었다.” “AI가 생성한 테스트 코드의 커버리지가 실제로 어느 정도인지 검증해봤다.” “Cursor와 Copilot을 같은 작업에 적용해봤더니 이런 차이가 있었다.” 이런 이야기가 시장에서 가치 있는 소재가 된다. 아무나 할 수 있는 일이지만, 아무도 안 한다. 하는 사람이 엿지를 가져간다.

Claude Code를 안 쓰는 것 자체가 뒤쳐지는 것이다. 그런데 “Claude Code 씁니다”만으로는 차별화가 안 된다. 차별화는 “Claude Code로 이걸 해봤는데, 이런 문제를 발견했고, 이렇게 해결했다”에서 나온다. 경험의 깊이가 엿지다.

어떤 도구를 어떤 상황에서 쓰고, 프롬프트를 어떻게 설계하고, AI와 협업하는 워크플로우를 어떻게 만드는지 — 구체적인 실행법은 「AI를 내 편으로 만들고 10년을 따라잡는 법」에서 더 깊이 다룬다. 이 책이 “왜”와 “방향”을 말한다면, 그 책은 “어떻게”와 “실전”을 말한다. 둘 다 있어야 완성이다.

시그니처 프로젝트가 이력서의 킷이 된다

이력서에 프로젝트가 세 개 있다고 치자. Spring Boot로 만든 게시판, 쇼핑몰 클론, TODO 앱. 면접관은 이걸 수백 번 봤다. 눈에 안 들어온다. 솔직히 말하겠다. 면접관은 이런 프로젝트를 3초 만에 넘긴다. “아, 또 게시판이구나.”

2026년에는 여기에 하나가 추가됐다. “AI로 만든 CRUD 앱.” 이것도 마찬가지다. 바이브코딩으로 똑딱 만든 앱은 “AI를 써봤습니다” 이상의 의미가 없다. AI를 써서 만들었다는 건, 누구나 할 수 있다는 뜻이니까.

시그니처 프로젝트는 다르다. **AI를 실제로 적용해서, 기획부터 배포, 운영, 유저 확보까지 전부 혼자서 해본 프로젝트.**

“혼자서 전부”가 핵심이다.

기획을 내가 했다. “이런 문제가 있고, 이걸 이렇게 풀겠다”를 내가 정했다. 기술 선택을 내가 했다. “이 상황에서 이 기술을 쓰는 이유는 이거다”를 내가 결정했다. 개발을 AI와 함께 했다. AI한테 시키되, AI의 결과물을 판단하고 수정하는 건 내가 했다. 배포를 내가 했다. 운영을 내가 했다. 유저를 내가 모았다. 피드백을 받고 개선을 내가 했다.

이 전체 사이클을 경험한 사람은 면접에서 할 이야기가 넘친다.

“왜 이 기술을 선택했어?” → “세 가지 대안을 비교했는데, 이런 이유로 이걸 선택했습니다.” “어려웠던 점은?” → “AI가 생성한 코드에서 이런 문제가 있었고, 직접 디버깅해서 이렇게 해결했습니다.” “사용자 반응은?” → “실제 유저 50명을 모았고, 이런 피드백을 받아서 이렇게 개선했습니다.” “다시 한다면?” → “이 부분을 다르게 설계했을 겁니다. 이유는 이렇습니다.”

매 질문에 자기 경험으로 답할 수 있다. 외운 게 아니라 겪은 거니까. 면접관은 이 차이를 즉시 느낀다.

2022년이라면 이력서의 킷은 “카프카 메시지 유실 문제를 해결했습니다”나 “동시성 이슈를 이렇게 풀었습니다” 같은 엔지니어링 난제였다. 2026년에는 다르다. **“AI를 활용해서 혼자서 프로젝트를 만들고, 실제 유저를 확보하고, 운영까지 해봤습니다”**가 킷이 된다.

왜냐하면 이게 곧 “이 사람은 시키지 않아도 알아서 돌아가는 사람이다”의 증거이기 때문이다. 문제를 정의하고, 해결하고, 배포하고, 운영하는 전체 과정을 경험한 사람. 회사는 그런 사람을 원한다. 특히 주니어한테서 그런 모습이 보이면 당장 뽑고 싶어진다.

구체적으로 어떻게 시작하냐고? 내 주변의 불편함에서 시작해라. 거창할 필요 없다. “우리 스터디 출결 관리가 불편하다” → AI로 출결 관리 봇을 만든다. “채용공고를 매번 하나씩 읽는 게 귀찮다” → AI로 채용공고 분석 도구를 만든다. “코딩테스트 풀 때 비슷한 유형인지 구분이 안 된다” → AI로 문제 유형 분류기를 만든다. 문제를 내가 정의하고, 해결을 내가 하는 것. 이 과정 자체가 시그니처다.

그리고 반드시 배포까지 해라. 로컬에서만 돌아가는 프로젝트는 프로젝트가 아니라 연습이다. 배포하고, 사람들에게 쓰게 하고, 피드백을 받아라. 그 과정에서 생기는 문제 — 서버 비용, 에러 핸들링, 사용자 경험 — 이런 것들이 진짜 경험이다. 이런 이야기를 면접에서 하면 면접관 눈이 달라진다.

세 축은 서로를 먹여 살린다

이 세 개의 축은 따로 노는 게 아니다.

엔지니어링 기초가 있으니까 AI를 깊게 쓸 수 있다. AI가 제안하는 코드를 판단할 수 있으니까. AI를 깊게 쓰니까 시그니처 프로젝트가 가능하다. 혼자서는 못 만들었을 것을 AI와 함께 만들 수 있으니까. 시그니처 프로젝트를 만드니까 엔지니어링 기초가 더 단단해진다. 실제 프로젝트를 운영하면서 이론으로만 알던 것들을 몸으로 겪으니까.

이 선순환이 돌기 시작하면 성장 속도가 달라진다. 세 축 중 하나만 하는 사람은 성장이 선형적이다. 세 축을 동시에 돌리는 사람은 성장이 기하급수적이다. 각 축에서 얻은 경험이 다른 축을 가속시키니까.

다만, 동시에 시작하되 비중은 시기에 따라 다르다. 내가 지금 신입이라면, 이렇게 돌릴 거다.

처음 두 달은 기초 체력에 하루의 절반 이상을 쓴다. CS, 알고리즘, 객체지향, 테스트 코드. 동시에 매일 30분은 AI 콘텐츠를 흡수한다. 트위터, 링크드인, 유튜브. 주 1회는 AI 도구를 직접 써본다. Claude Code로 간단한 기능을 만들어보거나, AI로 알고리즘 풀이를 검증해본다. 이 시기의 목표는 체력 만들기과 감 잡기다. 아직 뭔가를 만들 단계는 아니다.

3~4개월차부터 시그니처 프로젝트에 비중을 높인다. 앞에서 말한 불편함 중 하나를 골라서, 기획부터 AI와 함께 진행한다. 이 과정에서 부딪히는 기술적 문제가 곧 기초 공부의 방향이 된다. “캐시가 필요하다” → Redis를 공부한다. “비동기 처리가 필요하다” → 메시지 큐를 공부한다. 프로젝트가 공부의 방향을 정해준다. 동시에 블로그 첫 글을 쓴다. 완벽하지 않아도 된다. 시작하는 게 중요하다. 들어가며에서 말한 부트캠프 수료생도 이 시점에서 방향을 잡았다. 이것저것 흠뻑리던 에너지를 하나의 프로젝트에 모았더니 깊이가 생겼다.

5~6개월차에는 프로젝트를 배포하고, 실제 유저 30~50명을 모은다. 운영하면서 터지는 문제를 해결한다. 이 과정을 시리즈 3~5편으로 쓴다. 링크드인에 올리고, 커뮤니티에서 피드백을 받는다. 이력서를 쓰기 시작한다.

프로젝트에서 겪은 의사결정을 이력서에 녹인다. 채용공고를 다시 읽는다. 처음에 읽었을 때와 다르게 보일 거다. 이제 행간이 읽힌다. 타깃 회사 5곳 이상에 지원한다.

이 6개월은 예시일 뿐이다. 사람마다 속도가 다르다. 중요한 건 순서다. 기초 없이 프로젝트를 시작하면 무너지고, 프로젝트 없이 글을 쓰면 소재가 없다. 순서를 지키면서 세 축을 동시에 돌리는 것. 그게 핵심이다.

Part 4. 보이지 않으면 없는 것이다

세 개의 축을 돌리고 있다. 엔지니어링 기초를 매일 쌓고 있고, AI 엣지를 찾아서 시도하고 있고, 시그니처 프로젝트를 만들고 있다. 좋다. 근데 이걸 아무도 모른다면?

없는 것과 같다.

혼자 방에서 아무리 실력을 키워도, 그걸 밖에서 아무도 모르면 기회가 오지 않는다. 이력서를 넣어야만 기회가 생기는 구조에서, 이력서를 넣기 전에 이미 알려져 있는 사람은 다른 게임을 하는 거다.

글쓰기는 포지셔닝이다

블로그에 “오늘 공부한 것 정리”를 올리는 건 일기장이다. “Spring Security 설정 방법 정리”, “JPA N+1 문제 해결 방법.” 이런 글이 블로그에 50개 있어도 포지셔닝이 안 된다. 왜냐하면 이런 글은 인터넷에 수만 개가 있으니까. 검색해도 안 나오고, 나와도 눈에 띄지 않는다.

포지셔닝이 되는 글쓰기는 다르다. **시리즈로 쓴다. 깊이를 보여준다. 남들이 안 하는 질문을 던진다.**

예를 들어보겠다. 링크드인에 “이벤트 드리븐 아키텍처”를 주제로 5편짜리 시리즈를 연재한다고 해봐라.

1편: 왜 이벤트 드리븐인가 — 동기 방식의 한계에서 출발해서, 우리 프로젝트에서 어떤 문제가 있었는지, 왜 이 방식을 선택했는지를 쓴다.

2편: Kafka vs RabbitMQ vs SQS — 뭘 선택할 것인가. 세 가지를 실제로 비교해보고, 각각의 장단점, 우리 상황에서 왜 이것을 선택했는지, 선택하지 않은 것들은 왜 탈락했는지를 쓴다.

3편: 실제 구현 — AI를 활용해서 어떻게 만들었는가. Claude Code로 기본 구조를 잡고, 어디서 AI의 도움을 받았고, 어디서 직접 수정했는지를 쓴다.

4편: 운영에서 터진 문제들 — 배포 후에 발생한 장애, 예상 못 한 이슈, 어떻게 대응했는지를 쓴다. 실패 경험이 담긴 글은 특히 가치가 높다.

5편: 회고 — 전체 과정을 돌아보며, 다시 한다면 뭘 다르게 할 것인가를 쓴다.

이 5편을 읽은 면접관은 생각한다. “이 사람은 이벤트 드리븐에 대해 깊이 파봤구나. 직접 만들어봤구나. 문제를 만나봤구나. AI도 실전에서 써봤구나. 그리고 그걸 정리할 줄 아는 사람이구나.”

글 하나는 잊힌다. 시리즈는 남는다. 그리고 시리즈를 쓴 사람은 그 주제의 전문가로 인식된다. 실제로 전문가가 아니어도. 왜냐하면 시리즈를 쓸 만큼 파고든 사람이 그 분야에서 거의 없기 때문이다. 대부분은 “카프카 써봤습니다”에서 멈춘다. 5편짜리 시리즈를 쓴 사람은 그 자체로 상위 1%다.

나도 이 방법을 썼다. 블로그에 글을 쓰기 시작하면서 내 이름이 알려졌다. 처음에는 아무도 안 읽었다. 당연히. 하지만 계속 쓰다 보니 사람들이 모였다. 면접관이 “블로그 봤습니다”라고 말한 적도 있다. 채용 담당자가 먼저 연락한 적도 있다. 글쓰기는 내 커리어에서 가장 효율 높은 투자였다. 코딩 실력은 면접에서만 보이지만, 글은 면접 전에 이미 보인다.

그런데 여기서 중요한 건, 대충 쓰지 마라는 거다. 글도 프로덕트다. 제목이 중요하다. 구조가 중요하다. 읽는 사람이 “다음 편도 읽어야겠다”고 느끼게 만드는 게 중요하다. 글을 쓸 때도 “이 글의 타겟은 누구인가, 이 글을 읽고 뭘 얻어가야 하는가”를 먼저 정하고 써라. 그게 포지셔닝이다.

사람 사이에서 검증되어야 진짜다

혼자 만든 무기는 혼자만 안다. 그리고 혼자만 알면 착각이 생긴다. “난 잘하고 있어”라는 착각. 이 착각이 위험한 이유는, 시장의 피드백을 늦게 받기 때문이다. 6개월 동안 열심히 했는데, 면접에서 “이건 좀...” 하는 피드백을 받으면 그때부터 다시 시작이다. 그 6개월을 돌이킬 수 없다.

스터디에서 코드 리뷰를 주고받아라. 내가 만든 코드를 다른 사람한테 보여주고, 피드백을 받아라. 내가 쓴 글을 같이 읽고, 뭐가 부족한지 들어라. 내가 만든 프로젝트를 시연하고, 질문을 받아라. 이 과정이 면접의 리허설이 된다.

피드백을 받았을 때 방어적으로 나오는 사람이 있다. “아, 이건 제가 일부러 이렇게 한 건데요...” 이런 사람은 면접에서도 방어적이다. 반면에 “아 맞네, 이 부분 놓쳤다. 고마워” 하고 바로 수정하는 사람이 있다. 이 사람은 면접에서도 자연스럽다. “그 부분은 미처 고려하지 못했는데, 좋은 지적입니다. 이렇게 개선할 수 있을 것 같습니다.”

다른 사람의 코드를 읽으면서 건설적인 의견을 주는 사람이 있다. “이 부분은 이렇게 하면 성능이 좀 더 나을 것 같은데, 어떻게 생각해?” 이런 사람은 팀에서 환영받는다. 반면에 “좋은 것 같아요”로 끝내는 사람이 있다. 이 사람은 코드 리뷰 경험이 있어도 깊이가 없다.

결국 취업은 “**이 사람이랑 같이 일하면 어떨까**”로 결정된다. 수많은 상담을 하면서 확신하게 된 것이 있다. 잘 되는 사람의 진짜 공통점은 기술력이 아니다. “**이 사람이랑 같이 일하고 싶다**”가 느껴지는 사람이다. 똑부러지고, 본인 생각이 있고, 게으르지 않고, 대화가 되는 사람. 이건 연기로 되는 게 아니다. 실제로 사람들 사이에서 그렇게 행동해온 경험이 쌓여야 자연스럽게 나온다.

그래서 어디서 사람을 만나느냐가 중요하다.

인프런, OKKY 같은 대형 개발자 커뮤니티에서 먼저 시작해라. 질문을 올리되, “이거 어떻게 해요?”가 아니라 “이렇게 했는데, 이 접근이 맞는지 검증받고 싶습니다”로 올려라. 답변도 달라라. 남의 질문에 답을 다는 과정에 서 내가 뭘 아는지, 뭘 모르는지가 선명해진다.

오프라인 스터디도 필수다. 다만 하나 주의할 게 있다. **같은 처지의 취준생만 만나지 마라.** 스터디를 하는데 전원이 비전공 취준생이면, 서로 모르는 걸 물어보다가 결국 “그런 것 같다”로 끝난다. 시장과 단절된 상태에서 서로 의 불안만 공유한다. 이건 스터디가 아니라 위로 모임이다.

현업자가 한 명이라도 있는 스터디를 찾아라. 코드 리뷰를 받되, 시니어한테 받을 수 있는 기회를 만들어라. 고립은 성격의 문제가 아니라 전략의 부재다.

내가 운영하는 카카오톡 오픈채팅방도 그런 공간 중 하나다. 현재 400명 가까이 모여 있고, 자유롭게 스터디를 모집하고, 정보를 공유하고, 서로 코드 리뷰를 해준다. 현업자도 있고, 취준생도 있다. 이런 환경에 자신을 노출시키는 것 자체가 검증의 시작이다.

사람 사이에서 검증받는 과정이 결국 “함께 일하고 싶은 사람”의 증거가 된다. 그리고 그 과정에서 만난 사람들이 나중에 기회를 연결해주는 경우도 많다. 추천, 내부 채용 정보, 멘토링. 이건 혼자 방에서 공부하면 절대 얻을 수 없는 것들이다.

이력서는 일기장이 아니라 마케팅 문서다

이력서에 “~를 했습니다”만 나열하면 일기장이다. 면접관은 네가 뭘 했는지가 아니라, **네가 뭘 할 수 있는지를** 알고 싶다.

내가 이력서 합격률을 1%에서 66%로 바꾼 건, 내용을 바꾼 게 아니라 관점을 바꾼 거다.

Before: “Spring Boot와 JPA를 사용하여 게시판을 만들었습니다.”

이걸 읽는 면접관의 반응: “음, 그래서? 수천 명이 똑같은 걸 했는데.”

After: “사용자 조회 성능을 개선하기 위해 N+1 문제를 분석하고, Fetch Join과 Batch Size 전략을 비교한 뒤 Batch Size를 선택했습니다. 선택 이유는 쿼리 수를 줄이면서도 불필요한 데이터 로딩을 방지할 수 있었기 때문입니다.”

이걸 읽는 면접관의 반응: “오, 이 사람은 문제를 인식했고, 대안을 비교했고, 근거를 가지고 선택했구나. 생각하는 사람이네.”

같은 프로젝트다. 같은 경험이다. 하지만 후자에는 의사결정이 보인다. 왜 그렇게 했는지, 어떤 대안이 있었는지, 왜 이걸 선택했는지. 면접관은 이걸 보고 “이 사람은 생각하는 사람이구나”를 느낀다.

2026년 이력서에는 한 가지가 더 필요하다. **AI와 어떻게 협업했는지.**

“Claude Code를 활용해서 이벤트 드rib 아키텍처를 구현했습니다. AI가 제안한 초기 구조에서 메시지 순서 보장 부분을 수정했습니다. 이유는 우리 비즈니스 로직에서 주문 생성→결제→배송의 순서가 보장되어야 했기 때문입니다.”

이런 맥락이 있으면 면접관은 확신한다. 이 사람은 AI를 도구로 쓸 줄 안다. 시키면 하는 게 아니라, AI를 시킬 줄 아는 사람이다. AI의 결과물을 판단하고 수정할 줄 아는 사람이다.

이력서를 쓸 때 반드시 이 질문을 해라. **“이걸 읽는 면접관이, 나와 함께 일하는 모습을 상상할 수 있는가?”** 상상이 안 되면 다시 써라. 구체적인 상황, 구체적인 판단, 구체적인 결과가 있어야 상상이 된다. 추상적인 “열심히 했습니다”로는 그림이 안 그려진다.

이력서는 내가 한 일의 기록이 아니다. **나를 뽑아야 하는 이유를 설득하는 문서다.** 그 관점으로 다시 써봐라. 달라진다. 그리고 이력서가 달라지면, 면접도 달라진다.

면접은 이력서의 연장선이다

이력서가 마케팅 문서라면, 면접은 프레젠테이션이다. 같은 제품을 피칭하는 거다. 이력서에서 “이 사람이 뭘 할 수 있는지”를 보여줬으면, 면접에서는 그걸 직접 증명하는 자리다.

면접에서 떨어지는 사람의 공통점이 있다. 질문에 “답”을 하려고 한다. “JPA N+1 문제를 설명해보세요” → 외운 답을 말한다. “프로젝트에서 어려웠던 점은?” → “일정이 빡빡했지만 열심히 했습니다.”

면접에서 붙는 사람은 다르다. 질문에 “경험”을 말한다. “N+1 문제를 프로젝트에서 실제로 만났는데, 쿼리 로그를 보고 발견했습니다. Fetch Join과 Batch Size를 비교했고, 이런 이유로 이걸 선택했습니다.” 이건 외운 게 아니라 겪은 거다. 면접관은 이 차이를 즉시 알아챈다.

기술 면접의 핵심은 깊이다. “~를 알고 있느냐”가 아니라 “~를 써봤을 때 어떤 문제를 만났고, 어떻게 판단했느냐”를 묻는다. 시그니처 프로젝트가 있으면 이 질문에 전부 자기 경험으로 답할 수 있다. 프로젝트가 곧 면접 준비다.

인성 면접의 핵심은 진정성이다. “본인의 강점이 뭔가요?”에 “성실함입니다”라고 답하면 아무 정보가 없다. “매일 새벽 6시에 일어나서 1시간 동안 알고리즘을 풀었고, 6개월 동안 하루도 빠지지 않았습니다. 그 루틴이 이 프로젝트를 완성하게 만든 힘이었습니다.” 이러면 성실함이 증명이 된다.

이력서 작성법, 면접 전략, 합격률을 바꾸는 구체적인 기술에 대해서는 내가 따로 쓴 글들이 있다. 이 책에서 다루기에는 범위가 넓은 내용이라, 관심 있다면 찾아보길 권한다. 여기서는 방향만 잡아준다.

면접은 연기하는 자리가 아니다. 이력서에 쓴 것을 증명하는 자리다. 이력서에 거짓이 없으면, 면접에서 떨 이유도 없다. 시그니처 프로젝트를 만들고, 글로 정리하고, 사람 사이에서 검증받은 사람은 면접에서도 자연스럽다. 왜냐하면 전부 겪은 이야기를 하는 거니까.

그런데, 뭔가 빠졌다

솔직히 말하겠다.

여기까지 쓰고 나서, 마음이 석연치 않았다.

판을 읽어라. 시스템을 만들어라. 무기 세 개를 돌려라. 글로 증명하고, 사람 사이에서 검증받아라. 다 맞는 말이다. 근데 이걸 다 모아놓고 보니, 뭔가 마지막 한 조각이 비어 있는 느낌이었다.

정말 이게 전부일까? 정말 내가 지금 신입이라면, 이 방식으로 확신을 가질 수 있을까? 더 절박하게, 더 선명하게 베풀 수 있을까?

이게 그냥 “3년 차 선배의 조언 모음집”이 되면 안 되는데. 서점에 널린 그런 책이면 안 되는데. 뭔가 하나, 이 모든 것을 꿰뚫는 게 있어야 하는데.

그래서 진짜로 신입이 되어보려고 했다. 지금의 경험과 지식을 다 내려놓고, 2026년에 처음으로 개발 공부를 시작하는 사람의 머릿속으로 들어가보려고 했다.

한참을 고민했다.

처음엔 이렇게 생각했다. “결국 나만의 무기를 찾는 게 핵심 아닌가?” 내 경우에는 글쓰기가 무기였다. 블로그를 쓰면서 문제에 집착하고, 딥다이브하고, 남들이 안 하는 질문을 하는 포지셔닝이 됐다. 그러면 이 책에서도 “나만의 집착 포인트를 찾아라”고 쓰면 되지 않나?

근데 아니었다. 솔직히 생각해 보면, 대부분의 신입은 집착 포인트 같은 게 없다. 뭘 좋아하는지도 아직 모르고, 뭘 잘하는지도 모르고, 그냥 취업하고 싶은 거다. “집착 포인트를 찾아라”는 나한테 맞았던 거지, 보편적인 답이 아니었다.

그러면 뭘까? 그 많은 상담에서 봤을 때, 진짜 잘 되는 사람의 공통점이 뭐였나?

“이 사람이랑 같이 일하면 어떨까가 그러지는 사람.” 면접에서도 그렇고, 스터디에서도 그렇고, 멘토링에서도 그렇다. 똑부러지고, 본인 생각이 있고, 게으르지 않은 사람. 이걸 맞다. 근데 이건 결과지, 방법이 아니다. “그런 사람이 되어라”라고 쓸 수는 있지만, 그건 “키 크세요”랑 비슷한 말이다. 어떻게?

좀 더 파봤다. 안 되는 사람의 공통점을 뒤집어보면 답이 나오지 않을까?

안 되는 사람의 패턴은 명확했다. 주어에 “내”가 없다. 남들 하는 거 따라한다. 아는 건 많은데 선택한 게 없다. 공부는 하는데 결정을 안 해본다. CS 공부하고, 코딩테스트 풀고, 이력서 내고. 이 루프만 돈다. 이 루프 안에 “내가 뭘 만들겠다”, “내가 이 방향으로 가겠다”는 결정이 없다.

그래, 의사결정이다. 의사결정의 흔적이 있는 사람. 맞는 말이었다.

근데 한 발 더 들어가면, 의사결정은 결과물이지 원천이 아니었다. 의사결정을 하게 만드는 것. 그 이전에 있는 것. 그게 뭔데?

무기도 있고, 루틴도 있고, 글도 쓰고, 사람도 만나고, 의사결정도 한다. 다 맞는데. 이것을 관통하는 하나가 뭔데?

머칠을 더 끝냈다. 자기 전에도 생각하고, 출근길에도 생각하고, 멘티랑 상담하면서 이 질문이 머릿속에서 안 떠났다. “이 모든 걸 한 단어로 말하면 뭔데?”

그러다 어느 순간, 떨어졌다.

이거였다.

“스스로 자립하지 못하면, 회사 역시도 나를 필요로 하지 않는다.”

자립.

2026년에 시가 “시키면 하는 사람” 역할을 대체했다. 그러면 회사가 필요로 하는 건? 시키지 않아도 스스로 돌아가는 사람이다. 문제를 스스로 정의하고, 방향을 스스로 잡고, 끝까지 스스로 만들어내는 사람.

그리고 돌아보니, 내가 여기까지 쓴 모든 것이 결국 “자립”을 말하고 있었다.

판을 읽는 것도 스스로 하는 거다. 시스템을 만드는 것도 스스로 하는 거다. 무기를 쌓는 것도, 글을 쓰는 것도, 사람 사이에서 검증받는 것도, 전부 스스로 하는 거다. 누가 시켜서 하는 게 아니라, 내가 판단하고 내가 실행하는 거다.

그게 빠져 있었다. 이 모든 것을 관통하는 한 단어. 전략이 아니라 태도. 스킬이 아니라 존재 방식.

이제 마지막 이야기를 할 수 있겠다.

Part 5. 스스로 돌아가는 사람

회사가 더 이상 필요로 하지 않는 사람의 조건

자립이라는 단어가 떨어지고 나니, 시야가 선명해졌다. 왜 이게 2026년의 핵심인지, 이제 정확하게 설명할 수 있다.

예전에는 “시키면 잘 하는 사람”이 쓸모 있었다. 주어진 티켓을 정확히 처리하고, 요구사항대로 구현하고, 기한 안에 납품하는 사람. 성실한 실행자. 회사는 그런 사람을 10명 모아서 프로젝트를 돌렸다. 그리고 그게 주니어의 역할이었다. “시키는 걸 빠르고 정확하게 처리하는 것.” 그걸 잘하면 인정받았다.

이제 그 역할을 AI가 한다.

티켓을 주면 코드를 짜준다. 요구사항을 알려주면 구현해준다. 에러를 보여주면 고쳐준다. 테스트 코드도 작성해준다. 문서화도 해준다. 그것도 불평 없이. 24시간. 휴일 없이. 야근 수당도 안 받고.

그러면 회사가 더 이상 필요로 하지 않는 사람은 누구인가?

시켜야만 움직이는 사람이다.

“뭐 하면 될까요?”를 매번 묻는 사람이다. 할 일을 받아서 처리하고, 다 끝나면 다시 “다음 뭐 할까요?”를 묻는 사람이다. 지시가 없으면 멈추는 사람이다.

이건 성실함의 문제가 아니다. 구조의 문제다. AI가 실행을 대체하는 시대에, 실행만 잘하는 사람은 대체 가능하다. 아무리 성실해도, 아무리 빨라도, AI보다 빠르지는 않다. AI보다 싸지도 않다. 성실한 실행력으로 승부하려는 전략 자체가 2026년에는 작동하지 않는다.

AI가 대체한 것, 대체하지 못한 것

AI가 대체한 건 명확하다. 코드 작성, 디버깅, 문서화, 간단한 설계, 반복적인 리팩토링, 테스트 코드 생성. “이걸 이렇게 만들어줘”라고 하면 만들어준다. “이 코드에 버그가 있어, 찾아줘”라고 하면 찾아준다. “이 함수에 대한 단위 테스트를 작성해줘”라고 하면 작성해준다.

AI가 대체하지 못하는 건 이거다.

“이걸 왜 만들어야 하는지”를 정하는 것.

어떤 문제가 진짜 문제인지 판단하는 것. 수많은 이슈 중에서 지금 가장 중요한 게 뭔지 우선순위를 매기는 것. 여러 방향 중 어디로 갈지 결정하는 것. 이해관계자를 설득하는 것. 기획자가 원하는 것과 기술적으로 가능한 것 사이에서 최적의 지점을 찾는 것. 예상치 못한 상황에서 방향을 수정하는 것. 끝까지 책임지고 완성하는 것.

이건 전부 “스스로”의 영역이다. 시킨 사람이 없다. 매뉴얼이 없다. 정답이 없다. 그런데도 움직여야 한다.

2026년에 회사가 주니어한테 기대하는 건 달라졌다. “코드를 잘 짤 줄 아는 사람”이 아니라, **“혼자서 문제를 정의하고, 방향을 잡고, AI를 도구로 써서 끝까지 만들어내는 사람.”**

주니어한테 이 정도를 기대하냐고? 기대한다. 왜냐하면 기대하지 않으면 AI를 쓰면 되니까. 회사가 주니어를 뽑는 이유가 “시킬 일이 있어서”가 아니라, “성장해서 팀에 가치를 더할 수 있는 사람이니까”로 바뀌었다. 그리고 성장 가능성의 가장 강한 시그널은 “이미 스스로 움직여본 경험”이다.

결다리 — 정말 아무것도 안 되면, 나는 이렇게 한다

모든 사람에게 하는 이야기가 아니다. 이력서를 수십 군데 넣었고, 면접도 봤고, 포트폴리오도 만들었는데 아무 데서도 안 되는 사람. 진짜로 다 해봤는데 막혀 있는 사람. 그런 사람한테만 하는 이야기다.

내가 그 상황이라면, 남은 2년을 이거 하나에 건다. 소프트웨어 개발 자체.

취업이 아니다. 사업이다. 정확히는, 서비스를 만들고 운영하는 사람이 되는 거다.

구체적으로 이렇게 한다.

부모님 집으로 들어간다. 주거비를 0으로 만든다. 자존심 버려라. 지금 자존심 챙길 상황이 아니다.

동네에서 아르바이트를 잡는다. 스타벅스가 이상적이다. 잘 안 자르고, 시간이 유연하다. 목적은 하나다. 최소한의 생활비, 24시간 스터디카페 월 이용권, Claude Code Max 구독료. 이 세 가지만 벌면 된다. 아르바이트에 쓰는 시간은 최소로 줄인다. 먹고 자는 것 빼고 남은 시간 전부를 서비스 런칭에 쓴다.

여기서 핵심이 있다. 코딩에만 매달리면 안 된다. 실제로 코딩은 전체의 20~30%밖에 안 된다. 나머지는 이거다. 어떤 서비스를 만들지 정하는 것. 시장을 조사하는 것. 기획하는 것. 브랜딩하는 것. 마케팅하는 것. 아키텍처를 설계하는 것. 인프라에 배포하는 것. 운영하는 것. 이 전체를 혼자 한다. AI 에이전트를 도구로 써서, 혼자서 전부 돌린다.

매일 2시간은 무조건 기록한다. 오늘 뭘 했고, 뭘 배웠고, 뭘 실패했는지. 기록 없이 한 작업은 증발한다.

주 1회, 깊고 밀도 있는 글을 한 편 쓴다. 블로그와 링크드인에 올린다. 빠지지 않는다. 한 주도 빠지지 않는다. 52주면 52편이다. 52편의 밀도 있는 글이 쌓이면 그 자체가 포트폴리오가 된다.

1년 목표. 서비스 최소 3~5개 런칭. 잘 돌아가면 20개도 가능하다. 서비스당 유저 최소 50명. 어떻게든 모은다. 지인한테 부탁하든, 커뮤니티에 올리든, 카페에서 QR코드를 돌리든. 50명이 쓰는 서비스를 만들어 봤다는 건, “이 사람은 혼자서 처음부터 끝까지 돌릴 수 있는 사람이다”의 증거다.

백캐스팅을 건다. 1년 뒤의 상태를 먼저 정한다. “나는 서비스 5개를 운영하고 있고, 블로그에 글이 52편 있고, 링크드인 팔로워가 500명이다.” 거기서 역추적한다. 6개월 뒤에는 어디까지 와 있어야 하는가. 3개월 뒤에는. 이번 달에는. 이번 주에는. 오늘은.

1년 뒤에 이걸 들고 가면, 어떤 면접관이라도 멈춘다. 서비스 3~5개를 만들고, 유저를 모으고, 운영하고, 그 과정을 52편의 글로 기록한 사람. “이 사람은 시키지 않아도 돌아가는 사람이다”를 증명할 재료가 넘친다.

이게 쉽다는 말이 아니다. 극단적인 방법이다. 하지만 정말 아무것도 남지 않았다면, 이게 가장 확실한 베팅이다. 취업 시장에서 “뽑아주세요”를 반복하는 대신, “나는 이미 돌아가고 있는 사람이다”를 보여주는 것. 그게 2026년에 바닥에서 올라오는 유일한 방법이다.

이 책에서 말한 모든 것은 결국 여기로 수렴한다

시장을 읽는 것도, 루틴을 만드는 것도, 무기를 쌓는 것도, 증명하는 것도 — 전부 같은 근육이다. 누가 시키기 전에 스스로 움직이는 근육. 이 책에서 말한 모든 것은 결국 이 한 가지를 다른 각도에서 본 것이다.

내가 3년 전에 했던 것도 결국 전부 “스스로”였다. 누가 시킨 게 아니었다. 그걸 이제야 한 단어로 말할 수 있게 된 거다.

스스로 자립하지 못하면, 회사 역시도 나를 필요로 하지 않는다.

이게 2026년의 현실이다.

그런데 이건 절망적인 이야기가 아니다. 오히려 희망적이다.

왜냐하면 스스로 돌아가는 능력은 학벌이나 스펙이 아니라 습관이기 때문이다. 서울대를 나와야 할 수 있는 게 아니다. 대기업 인턴을 해야 배울 수 있는 게 아니다. 매일의 루틴에서 만들어지는 것이다. 오늘부터 시작할 수 있는 것이다.

나도 영문학과 철학을 전공한 문과 출신이다. 코드를 한 줄도 못 짜던 사람이다. 그런 내가 할 수 있었던 건, 스스로 움직였기 때문이다. 환경을 바꾸고, 루틴을 만들고, 시장을 읽고, 무기를 만들고, 증명했다. 전부 스스로.

이 책은 방향을 말한다. 무엇을 왜 해야 하는지. 하지만 방향만으로는 부족하다. 실제로 AI를 내 무기로 만드는 구체적인 실행법이 필요하다.

그래서 이 책은 「AI를 내 편으로 만들고 10년을 따라잡는 법」과 함께 나온다. 이 책이 “왜 움직여야 하는가”라면, 그 책은 “어떻게 움직이는가”다. 방향 없는 실행은 헛수고고, 실행 없는 방향은 공상이다. 둘 다 있어야 한다.

지금 이 책을 덮고, 채용공고를 하나 열어봐라. 행간을 읽어봐라. 그리고 6개월 뒤의 나를 정해봐라. 거기서부터 역추적해서 이번 주에 할 일을 정해봐라. 종이 한 장에 적어라. 폰 메모장이 아니라 종이에. 그걸 모니터 옆에 붙여라. 그리고 내일 아침, 1시간만 일찍 일어나봐라.

그게 첫 번째다. 나머지는 반복이다.

2026년에 살아남는 신입은, 시키기 전에 스스로 돌아가는 사람이다.

지금 시작해라.

만약에 내가 지금 신입이라면. 나는 이미 움직이고 있을 것이다.